BANCO DE DADOS · MODELAGEM CONCEITUAL

Aeroporto-BD

Modelo conceitual explicado: as entidades, os relacionamentos, suas cardinalidades e o porquê de cada escolha de modelagem.

Sistema de gestão do Aeroporto Internacional do Recife

Mariana Maliu · Arthur Coelho · Miguel Tojal · Raul Maia · Arthur Queiroz · Vitor Gadelha

github.com/mtojald/Aeroporto-BD

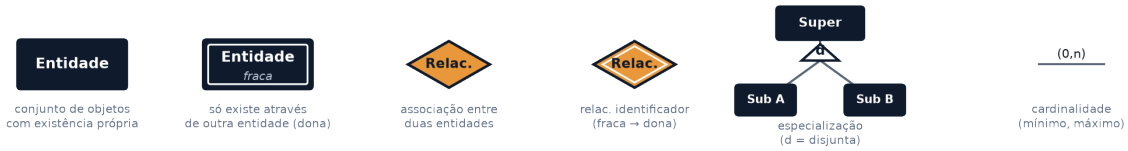
FUNDAMENTOS

O que é o modelo conceitual

O modelo conceitual é o primeiro passo do projeto de banco de dados: antes de pensar em tabelas, colunas ou SQL, ele descreve **o que existe no mundo real** que o sistema precisa guardar — voos, aeronaves, passageiros — e **como essas coisas se relacionam entre si**. É um desenho independente de qualquer banco de dados específico: só depois ele é traduzido para o modelo lógico (tabelas) e para o esquema relacional (o `CREATE TABLE` do MySQL).

Este documento usa a notação de Peter Chen, a mesma do diagrama original do projeto (`Projeto BD - Aeroporto.pdf`), e foca exatamente no que o nome diz: as entidades, os relacionamentos entre elas e as cardinalidades que definem as regras de negócio. As cardinalidades foram conferidas uma a uma contra as chaves estrangeiras e restrições do banco já implementado (`database/migrations.sql`), para garantir que o que está descrito aqui é exatamente o que o sistema impõe.

Notação usada nos diagramas



Como ler uma cardinalidade (mín., máx.)

Cada linha que liga uma entidade a um relacionamento carrega um par **(mínimo, máximo)**. Esse par fica do lado da entidade e responde: *"em quantas instâncias desse relacionamento UMA instância desta entidade pode aparecer?"*

- **(1,1)** — participação obrigatória e única: toda instância participa exatamente uma vez. Ex.: todo **Bilhete** tem exatamente 1 **Passageiro** (ninguém compra um bilhete sem dono).
- **(0,1)** — participação opcional e única: no máximo uma vez, podendo ser zero. Ex.: um **Voo** tem no máximo 1 voo de conexão associado.
- **(0,n)** — participação opcional e múltipla: de zero a muitas vezes. Ex.: uma **Companhia Aérea** opera de 0 a n **Aeronaves**.

Lendo a relação **Opera** entre Companhia Aérea e Aeronave como exemplo completo: o lado de Aeronave traz **(1,1)** — cada aeronave pertence a exatamente uma companhia — e o lado de Companhia Aérea traz **(0,n)** — uma companhia pode operar zero ou várias aeronaves. Lida assim, a cardinalidade de cada lado sempre descreve a própria entidade ao lado da qual está escrita — essa é a convenção usada em todos os diagramas deste documento.

ENTIDADES

O que existe no mundo do aeroporto

O modelo tem 16 entidades, organizadas em quatro papéis conceituais diferentes. Essa classificação — forte, fraca, associativa, especialização — não é só um detalhe acadêmico: ela determina diretamente como cada uma vira tabela (se tem chave própria ou chave composta herdada, se cai em cascata quando o “dono” é apagado etc.).

Tipo	Entidades	O que significa
Forte	Aeroporto, Terminal, CompanhiaAerea, Aeronave, Funcionario, Passageiro, Voo	Tem existência e chave primária independentes — não depende de nenhuma outra entidade para existir
Fraca	Portão (de Terminal), Manutenção e Assento (de Aeronave), Bagagem (de Bilhete)	Não tem sentido sozinha: sua chave é composta (um identificador parcial + a chave da entidade dona)
Especialização	Piloto, Comissário, Mecânico (de Funcionário)	Subtipos de Funcionário com atributos próprios; compartilham a mesma chave do supertipo
Associativa	Bilhete	Resolve uma associação entre Passageiro, Voo e Assento, mas tem identidade e chave próprias

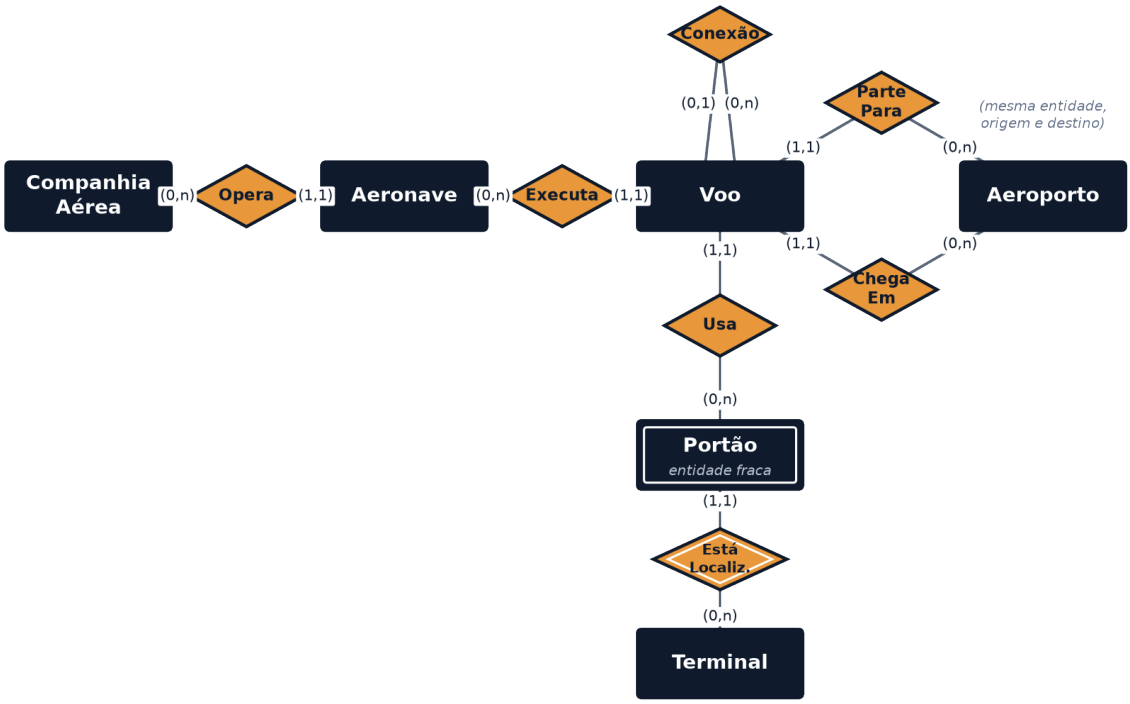
Por que isso importa

Uma entidade fraca sempre se conecta à sua dona por um **relacionamento identificador** (no diagrama, o diamante de borda dupla) — e é exatamente aí que mora a decisão de colocar `ON DELETE CASCADE` no banco: apagar a aeronave apaga os assentos dela, porque um assento não existe fora de uma aeronave.

ÁREA 1 · INFRAESTRUTURA E OPERAÇÃO

Companhias, aeronaves, voos e aeroportos

Este primeiro grupo modela o “esqueleto” da operação: quem é a companhia aérea, qual aeronave ela usa, qual voo essa aeronave realiza, de onde o voo parte, onde chega e qual portão ele ocupa.



Opera — CompanhiaAerea × Aeronave

Uma companhia aérea opera de 0 a n aeronaves **(0,n)**, e cada aeronave pertence a exatamente uma companhia **(1,1)**. É um relacionamento 1:N clássico: no banco, vira uma chave estrangeira obrigatória em `Aeronave`.

Executa — Aeronave × Voo

Cada voo é realizado por exatamente 1 aeronave **(1,1)**; uma aeronave realiza 0 a n voos ao longo do tempo **(0,n)**. Voo é o centro deste diagrama: quase tudo se conecta a ele.

PartePara e ChegaEm — Voo × Aeroporto (duas vezes)

Aeroporto participa **duas vezes** do mesmo voo — uma como origem, outra como destino — por isso aparecem dois relacionamentos distintos (PartePara e ChegaEm) entre as mesmas duas entidades. Em ambos, o voo tem exatamente 1 aeroporto do seu lado **(1,1)**, e um aeroporto é origem (ou destino) de 0 a n voos **(0,n)**. No banco, essa dupla participação se traduz em duas colunas de chave estrangeira em Voo (`fk_Aeroporto_origem` e `fk_Aeroporto_destino`), com uma restrição extra garantindo que elas nunca sejam iguais.

Usa / EstaLocalizado — Voo × Portão × Terminal

Um voo usa exatamente 1 portão de embarque **(1,1)**, e um portão pode ser usado por vários voos ao longo do tempo **(0,n)**. Portão, por sua vez, é entidade fraca de Terminal: só existe dentro de um terminal específico, por isso o relacionamento EstaLocalizado é **identificador** (diamante de borda dupla) — a chave do portão só faz sentido junto da chave do terminal.

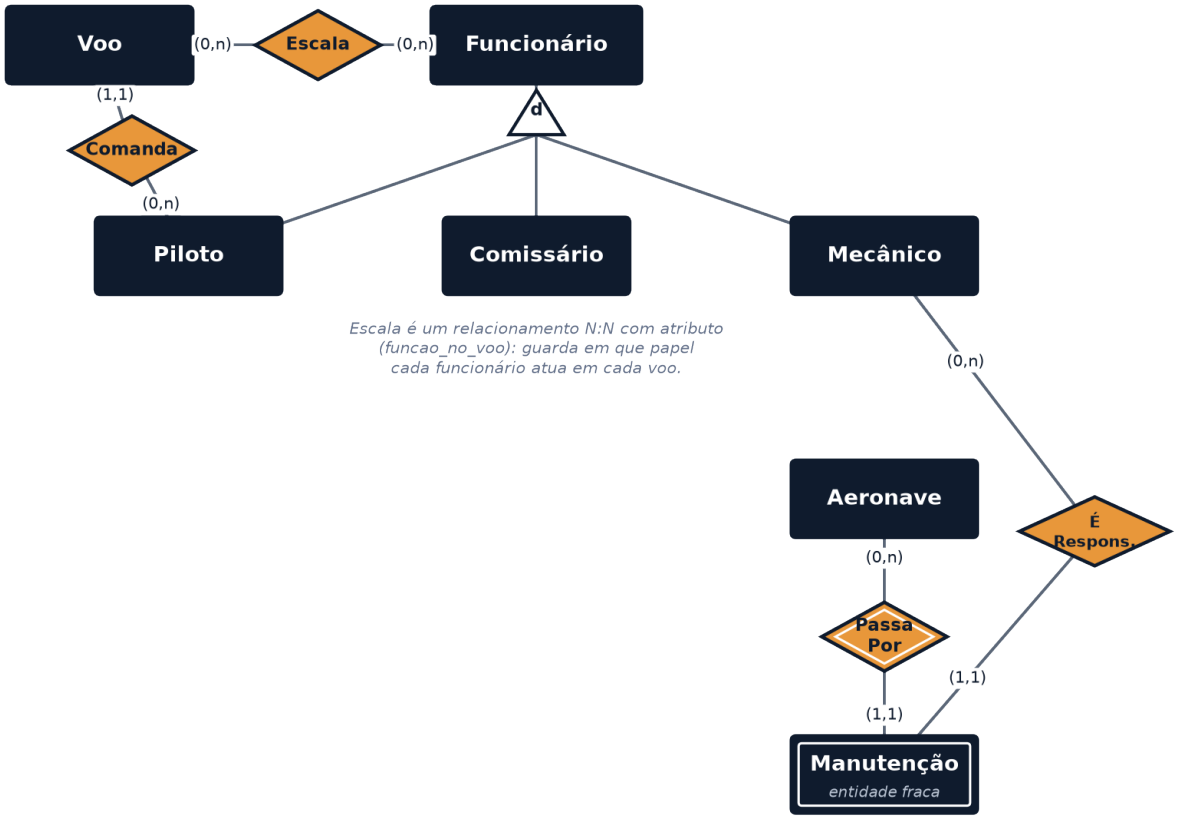
Conexão — auto-relacionamento de Voo

Voo se relaciona consigo mesmo: um voo pode ter, opcionalmente, um “voo de conexão” associado **(0,1)** — o próximo trecho da viagem. Olhando pelo outro lado, um voo pode ser a conexão de vários outros voos **(0,n)**, embora na prática isso normalmente aconteça no máximo uma vez. Esse é o único **auto-relacionamento** do modelo.

ÁREA 2 · TRIPULAÇÃO E MANUTENÇÃO

Funcionários, especialização e escala

Este grupo modela as pessoas que fazem o aeroporto funcionar — pilotos, comissários e mecânicos — e como elas se relacionam com voos e manutenções.



Especialização — Funcionário → Piloto / Comissário / Mecânico

Funcionário é a entidade genérica (matrícula, nome, CPF, endereço); Piloto, Comissário e Mecânico são **especializações** dela, cada uma com atributos próprios (licença e horas de voo; idiomas falados; especialidade técnica). O "**d**" no triângulo indica especialização **disjunta**: cada funcionário é, no máximo, um desses três papéis — não faz sentido a mesma pessoa ser piloto e mecânico ao mesmo tempo neste modelo. No banco, isso vira uma tabela por subtipo, cuja chave primária é também chave estrangeira para `Funcionario`.

Escala — Funcionário × Voo (N:N)

Um funcionário pode ser escalado em vários voos, e um voo tem vários funcionários escalados — por isso ambos os lados trazem **(0,n)**. É o único relacionamento N:N "puro" do modelo, e carrega um atributo próprio (`funcao_no_voo`: comissário chefe, copiloto etc.), o que obriga a virar uma tabela própria (`Escala`) em vez de uma simples chave estrangeira.

Comanda — Piloto × Voo

Diferente de Escala, Comanda é um relacionamento comum 1:N: cada voo tem exatamente 1 piloto comandante **(1,1)**, e um piloto pode comandar vários voos **(0,n)**. É por isso que “comandar” e “estar escalado” são coisas diferentes no modelo — um piloto comandante também poderia, em tese, aparecer em Escala com outro papel.

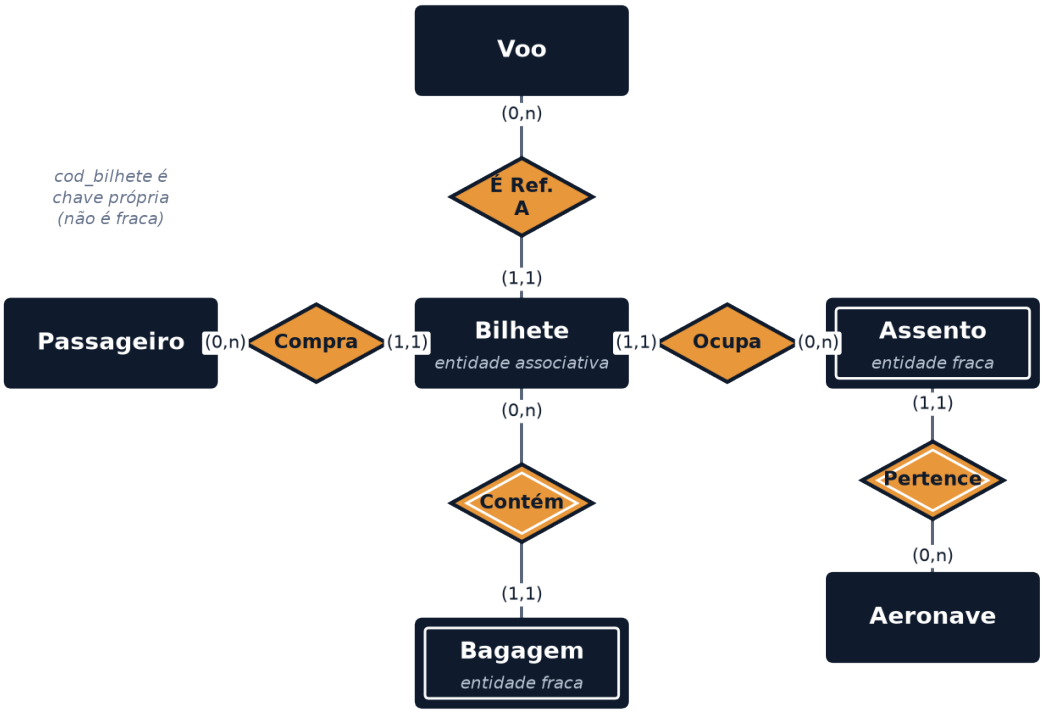
PassaPor e ÉResponsável — Manutenção

Manutenção é entidade fraca de Aeronave (PassaPor é relacionamento identificador: toda manutenção pertence a exatamente 1 aeronave **(1,1)**, e uma aeronave passa por 0 a n manutenções **(0,n)**). Separadamente, cada manutenção tem exatamente 1 mecânico responsável **(1,1)**, e um mecânico responde por 0 a n manutenções **(0,n)** — mas essa segunda relação NÃO participa da identidade da manutenção, por isso o diamante de É_Responsável não tem borda dupla.

ÁREA 3 · PASSAGEIROS, BILHETES E BAGAGEM

Do passageiro ao assento

O último grupo cobre a parte comercial: como um passageiro vira dono de um bilhete, como esse bilhete se conecta a um voo e a um assento específico, e como a bagagem se encaixa nisso.



Bilhete: entidade associativa, não entidade fraca

Bilhete é o ponto mais sutil do modelo. Ele **associa três entidades** — Passageiro, Voo e Assento — mas tem uma chave própria, independente (`cod_bilhete`), em vez de uma chave composta pelos três. Por isso ele é desenhado como entidade comum (forte), não como entidade fraca: um bilhete tem identidade própria mesmo sendo "sobre" outras três entidades.

Compra, ÉReferenteA e Ocupa

- **Compra** (Passageiro × Bilhete): um passageiro compra de 0 a n bilhetes **(0,n)**; todo bilhete pertence a exatamente 1 passageiro **(1,1)**.
- **ÉReferenteA** (Bilhete × Voo): todo bilhete é sobre exatamente 1 voo **(1,1)**; um voo tem 0 a n bilhetes vendidos **(0,n)**.
- **Ocupa** (Bilhete × Assento): todo bilhete ocupa exatamente 1 assento **(1,1)**; o mesmo assento físico pode ser ocupado por vários bilhetes ao longo de voos diferentes **(0,n)** — mas nunca duas vezes no

mesmo voo (essa regra mais fina é garantida por uma restrição `UNIQUE` envolvendo voo + assento, que já mistura o mundo lógico e foge do que um diagrama conceitual binário consegue expressar sozinho).

Pertence e Contém: as duas entidades fracas desta área

Assento é entidade fraca de Aeronave (cada assento só existe dentro de uma aeronave específica — por isso “12A” sozinho não identifica nada, precisa vir acompanhado da aeronave). Bagagem é entidade fraca de Bilhete (uma bagagem não existe sem um bilhete que a gerou). Em ambos os casos, o relacionamento identificador aparece com diamante de borda dupla.

ATRIBUTOS

Compostos e multivalorados

Além de entidades e relacionamentos, o enunciado do projeto pedia explicitamente dois tipos de atributo que fogem do “texto simples” comum: atributos **compostos** (que se dividem em partes) e **multivalorados** (que podem ter mais de um valor para a mesma entidade).

Tipo	Onde aparece	Como é modelado
Composto	Nome (Funcionario, Passageiro): primeiro_nome + sobrenome	Vira duas colunas simples na mesma tabela — não precisa de tabela própria
Composto	Endereço (Funcionario): CEP, rua, número, bairro	Mesma ideia: quatro colunas na tabela Funcionario
Multivalorado	Telefones de Funcionário, Passageiro e Companhia Aérea	Vira uma tabela filha (ex. Passageiro_Telefone) com a chave do dono + o telefone, podendo ter 0 a n linhas por dono
Multivalorado	Idiomas falados por um Comissário	Mesma ideia: tabela Comissario_Idioma, chave composta (matrícula + idioma)

A regra prática para multivalorado → tabela

Sempre que uma entidade pode ter **vários valores** para o mesmo atributo (um passageiro com dois telefones, por exemplo), o valor não cabe numa única coluna. A solução padrão é criar uma tabela filha cuja chave primária é a combinação (chave do dono + o próprio valor) — é exatamente o padrão repetido em `Passageiro_Telefone`, `Funcionario_Telefone`, `CompanhiaAerea_Telefone` e `Comissario_Idioma`.

RESUMO

Todos os relacionamentos, de ponta a ponta

Uma referência rápida com os 16 relacionamentos do modelo, suas cardinalidades exatas (conferidas contra as chaves estrangeiras do banco) e a regra de negócio que cada um impõe.

Relacionamento	Entidades	Cardinalidades	Regra de negócio
Opera	CompanhiaAerea × Aeronave	(0,n) / (1,1)	Toda aeronave tem 1 companhia dona
Executa	Aeronave × Voo	(0,n) / (1,1)	Todo voo usa exatamente 1 aeronave
PartePara	Voo × Aeroporto	(0,n) / (1,1)	Todo voo tem exatamente 1 aeroporto de origem
ChegaEm	Voo × Aeroporto	(0,n) / (1,1)	Todo voo tem exatamente 1 aeroporto de destino (≠ origem)
Usa	Voo × Portão	(0,n) / (1,1)	Todo voo usa exatamente 1 portão de embarque
EstaLocalizado	Portão × Terminal	(0,n) / (1,1)	Todo portão pertence a exatamente 1 terminal (fraca)
Conexão	Voo × Voo	(0,n) / (0,1)	Um voo tem no máximo 1 voo de conexão (auto-relac.)
Escala	Funcionário × Voo	(0,n) / (0,n)	N:N com atributo funcao_no_voo
Comanda	Piloto × Voo	(0,n) / (1,1)	Todo voo tem exatamente 1 piloto comandante
PassaPor	Aeronave × Manutenção	(0,n) / (1,1)	Toda manutenção pertence a 1 aeronave (fraca)
ÉResponsável	Mecânico × Manutenção	(0,n) / (1,1)	Toda manutenção tem exatamente 1 mecânico responsável
Compra	Passageiro × Bilhete	(0,n) / (1,1)	Todo bilhete pertence a exatamente 1 passageiro
ÉReferenteA	Bilhete × Voo	(0,n) / (1,1)	Todo bilhete é sobre exatamente 1 voo
Ocupa	Bilhete × Assento	(0,n) / (1,1)	Todo bilhete ocupa exatamente 1 assento
Pertence	Assento × Aeronave	(0,n) / (1,1)	Todo assento pertence a exatamente 1 aeronave (fraca)
Contém	Bilhete × Bagagem	(0,n) / (1,1)	Toda bagagem pertence a exatamente 1 bilhete (fraca)

FECHAMENTO

Do conceitual ao banco de dados real

Este modelo conceitual não ficou só no papel: ele foi a base direta do esquema relacional em `database/migrations.sql`. A tradução segue regras bem definidas:

No modelo conceitual	No banco (migrations.sql)
Entidade forte	Uma tabela, com chave primária própria
Entidade fraca	Uma tabela cuja chave primária é composta: identificador parcial + chave estrangeira da dona, com ON DELETE CASCADE
Especialização	Uma tabela por subtipo; a chave primária também é chave estrangeira para o supertipo
Relacionamento 1:N	Uma coluna de chave estrangeira na tabela do lado “N”
Relacionamento N:N (Escala)	Uma tabela associativa própria, com as duas chaves estrangeiras formando a chave primária composta
Atributo composto	Vira colunas simples na mesma tabela (sem tabela própria)
Atributo multivalorado	Vira uma tabela filha: chave do dono + o valor

Sobre este documento

Elaborado a partir do modelo conceitual original do projeto (`Projeto BD - Aeroporto.pdf`, Entrega 1) e verificado contra o esquema relacional efetivamente implementado em `database/migrations.sql`, para garantir que toda cardinalidade aqui descrita corresponde exatamente às restrições em vigor no banco.